

■

IdeaDNA

Guide 1: Idea Generator — Complete PRD

"Har Platform Ke Best Ideas + AI-Powered Upgrade"

Document:	Idea Generator — Feature PRD v1.0
Feature:	Core Mode 1 — Problem-to-Idea Generation
Date:	June 2026
Status:	Final — Ready for Development

1. Feature Overview

Idea Generator is the entry point of IdeaDNA. User koi problem ya theme describe karta hai, system uske liye **5-8 ranked idea cards** generate karta hai. Har card mein innovation score, feasibility score, aur difficulty level hota hai. User **sirf ek select** karta hai, uske baad hi deep-dive blueprint aur UI mockup generate hota hai (lazy-loading).

Key Differentiator: Har platform (Web, Mobile, AI, Blockchain, etc.) ke liye pre-loaded curated idea database hai. AI agent user ki specific need, skill level, budget, aur timeline ke hisaab se un ideas ko **real-time upgrade** karta hai.

2. User Flow & Interaction Design

- **Step 1 — Mode Selection:** User dashboard pe ■ 'Generate Ideas' select karta hai
- **Step 2 — Input:** Problem describe karta hai ya theme select karta hai (dropdown se bhi kar sakta hai)
- **Step 3 — Platform Filter:** User platform select karta hai: Web App, Mobile App, AI/ML, Blockchain, IoT, Desktop, Chrome Extension, API/Backend
- **Step 4 — Database Query:** System pehle curated database mein search karta hai matching ideas ke liye
- **Step 5 — AI Upgrade:** Retrieved ideas ko AI agent user ki profile ke hisaab se upgrade karta hai
- **Step 6 — Card Display:** 5-8 ranked cards show hote hain with scores, tags, and difficulty
- **Step 7 — Selection:** User ek card click karta hai → Deep-dive unlock hota hai
- **Step 8 — Deep Dive:** Blueprint + UI Mockup tabs appear (lazy-loaded generation)

3. Curated Idea Database

Har platform ke liye **pre-loaded, manually curated best ideas** stored hain. Ye database regularly update hoti hai based on trending technologies, market demand, aur hackathon winners.

Database Schema — CuratedIdeas Table

```
model CuratedIdea {
  id String @id @default(uuid())
  title String
  description String
  problemStatement String
  solution String
  keyFeatures String[] // Platform categorization
  platform Platform // WEB | MOBILE | AI_ML | BLOCKCHAIN | IOT | DESKTOP | EXTENSION | API
  subCategory String // e.g., "E-commerce", "Healthcare", "Education"
  // Quality metrics (manually rated by experts)
  innovationScore Int // 0-100
  marketPotential Int // 0-100
  complexity String // EASY | MEDIUM | HARD
  // Tech stack suggestions
  suggestedStack Json // {frontend: "React", backend: "Node", database: "PostgreSQL"}
  // Metadata
  tags String[]
  source String // "Hackathon Winner 2025", "YC Startup", "Trending 2026"
  trendingScore Int // 0-100 (auto-updated weekly)
  // Vector embedding for semantic search
  embedding Vector(1536)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}
enum Platform {
  WEB
  MOBILE
  AI_ML
  BLOCKCHAIN
  IOT
  DESKTOP
  EXTENSION
  API_BACKEND
}
```

Sample Curated Ideas (Per Platform)

Platform	Idea Title	Innovation	Market	Complexity
WEB	AI-Powered Resume Optimizer	88	92	MEDIUM
WEB	Real-time Collaborative Whiteboard	75	85	HARD
MOBILE	AR-based Furniture Placement	91	78	HARD
MOBILE	Voice-Enabled Expense Tracker	72	88	EASY
AI_ML	Medical Image Diagnosis Assistant	95	96	HARD
AI_ML	Smart Email Reply Generator	78	90	MEDIUM
BLOCKCHAIN	Decentralized Freelance Marketplace	85	82	HARD
BLOCKCHAIN	NFT-based Event Ticketing	80	75	MEDIUM
IOT	Smart Home Energy Optimizer	82	88	MEDIUM
IOT	Wearable Health Monitor for Elderly	88	92	HARD
DESKTOP	Offline-first Note-taking App	70	80	EASY
EXTENSION	AI Writing Assistant for Chrome	76	90	MEDIUM
API_BACKEND	Unified Payment Gateway API	80	95	HARD

Database Population Strategy

- **Initial Seed:** 500+ ideas manually curated from: Hackathon winners (Devpost, MLH), YC startups, Product Hunt trending, GitHub trending repos, Research papers

- **Weekly Update:** Cron job se trendingScore update hota hai based on: Google Trends API, GitHub stars growth, Product Hunt votes, Reddit r/SideProject mentions
- **Community Contribution:** Users apne winning ideas submit kar sakte hain (moderated)
- **Vector Embeddings:** Har idea ka OpenAI embedding store karte hain for semantic search

4. AI Agent Architecture — Need-Based Upgrade

AI agent ka kaam sirf new ideas generate nahi — **curated database se retrieved ideas ko user ki specific need ke hisaab se upgrade karna** hai. Ye 2-step process hai:

Step 1: RetrieverAgent — Database Se Ideas Lana

User ke input ko embed karke curated database mein semantic search karta hai. Top 10 matching ideas retrieve karta hai.

```
// RetrieverAgent Logic async function retrieveIdeas(userInput, platform,
userProfile) { // 1. Embed user input const embedding = await
openai.embeddings.create({ input: userInput, model: "text-embedding-3-small" }); //
2. Semantic search in curated database const matches = await prisma.$queryRaw`
SELECT id, title, description, innovationScore, 1 - (embedding <=>
${embedding.data[0].embedding}::vector) as similarity FROM "CuratedIdea" WHERE
platform = ${platform} ORDER BY similarity DESC, innovationScore DESC LIMIT 10 `;
// 3. Filter by user constraints return matches.filter(idea => { if
(userProfile.skillLevel === 'BEGINNER' && idea.complexity === 'HARD') return false;
if (userProfile.budget === 0 && idea.requiresPaidAPI) return false; return true;
}); }
```

Step 2: UpgradeAgent — User Need Pe Upgrade Karna

Retrieved ideas ko user ki profile ke hisaab se **modify, enhance, aur personalize** karta hai. Ye agent idea ko 'upgrade' karta hai — original concept same rehta hai, par implementation user ke constraints ke hisaab se adjust hota hai.

```
// UpgradeAgent System Prompt You are an Idea Upgrade Specialist. Your job is to
take a curated idea and upgrade it based on the user's specific needs, constraints,
and goals. Input: - Curated Idea: {original_idea} - User Profile: {user_profile} -
User's Specific Need: {user_need} Upgrade Rules: 1. KEEP the core concept and
innovation of the original idea 2. ADJUST the tech stack to match user's skills and
preferences 3. SCALE the complexity to match user's timeline and team size 4. ADD
features that specifically address user's stated need 5. REMOVE features that are
impossible given user's budget/constraints 6. ENHANCE with trending technologies if
user is advanced 7. SIMPLIFY if user is beginner Output: JSON with upgraded idea
containing: - title (modified if needed) - description (personalized) -
problemStatement (user's specific problem) - solution (adjusted for user's
constraints) - keyFeatures (added/removed based on need) - innovationScore (0-100,
may increase if upgrade is creative) - feasibilityScore (0-100, must match user's
constraints) - difficulty (EASY/MEDIUM/HARD, adjusted for user) - upgradeNotes
(what changed and why) - suggestedStack (personalized tech stack)
```

Upgrade Example

```
// Original Curated Idea { "title": "AI-Powered Resume Optimizer", "platform":
"WEB", "complexity": "MEDIUM", "suggestedStack": { "frontend": "React", "backend":
"Node", "ai": "OpenAI" } } // User Profile { "skillLevel": "BEGINNER", "budget": 0,
"teamSize": 1, "timeline": "2 weeks", "techPrefs": ["HTML", "CSS", "JavaScript"],
"need": "I want something simple that helps my college friends" } // Upgraded
Output { "title": "Simple Resume Checker for College Students", "description": "A
lightweight web app where students paste their resume and get instant feedback",
"problemStatement": "College students don't know if their resume is good enough",
"solution": "Paste resume → Get score + tips → Download improved version",
"keyFeatures": [ "Paste resume text (no upload needed)", "Instant score out of
```

```
100", "3 specific improvement tips", "Simple download as text" ],  
"innovationScore": 72, // Reduced because simplified "feasibilityScore": 95, //  
Increased because beginner-friendly "difficulty": "EASY", "upgradeNotes": [  
"Removed file upload (complex for beginners)", "Simplified AI to keyword matching  
instead of GPT analysis", "Added college-specific templates", "Removed payment  
integration (free tier only)" ], "suggestedStack": { "frontend": "HTML + CSS +  
Vanilla JS (no framework needed)", "backend": "None (client-side only)", "hosting":  
"GitHub Pages (free)" } }
```

5. Complete Agent Pipeline

1. InputParserAgent

User ke raw input ko structured format mein convert karta hai. Problem extract karta hai, platform detect karta hai, aur user ki implicit needs identify karta hai.

2. RetrieverAgent

Curated database mein semantic search karta hai. Platform filter + similarity score + quality score combine karke top 10 ideas retrieve karta hai.

3. RankerAgent

Retrieved ideas ko user profile ke hisaab se rank karta hai. Feasibility > Innovation > Trending score ka weightage deta hai.

4. UpgradeAgent

Top 5-8 ideas ko user ki specific need pe upgrade karta hai. Tech stack, complexity, features — sab personalize karta hai.

5. ScorerAgent

Har upgraded idea ko 4 dimensions mein score karta hai: Originality, Feasibility, Market Potential, Complexity Match.

6. OutputFormatter

Final output ko UI-ready JSON mein format karta hai. Cards ke liye data structure, colors, aur badges decide karta hai.

6. LangChain Implementation

```
// Complete LangChain Pipeline for Idea Generator
import { OpenAI } from "langchain/llms/openai";
import { LLMChain, SequentialChain } from "langchain/chains";
import { PromptTemplate } from "langchain/prompts";
import { StructuredOutputParser } from "langchain/output_parsers";

// === Agent 1: Input Parser ===
const inputParserPrompt = new PromptTemplate({
  template: `Analyze this user input and extract structured information:
  Input: {raw_input}
  Extract: 1. Core problem statement 2. Implied platform (web/mobile/ai/etc.) 3. Target audience 4. Key constraints mentioned 5. Emotional tone (urgent/casual/excited)
  Output: JSON with extracted fields.`,
  inputVariables: ["raw_input"]
});
const inputParser = new LLMChain({
  llm: new OpenAI({ temperature: 0.3 }),
  prompt: inputParserPrompt,
  outputKey: "parsed_input"
});

// === Agent 2: Upgrade Agent ===
const upgradePrompt = new PromptTemplate({
  template: `Upgrade this curated idea for the user:
  Original Idea: {original_idea}
  User Profile: {user_profile}
  Parsed Need: {parsed_input}
  Upgrade the idea while keeping core concept. Personalize for user's skill level, budget, timeline, and specific need. {format_instructions}`,
  inputVariables: ["original_idea", "user_profile", "parsed_input"],
  partialVariables: { format_instructions: upgradeParser.getFormatInstructions() }
});
const upgradeAgent = new LLMChain({
  llm: new OpenAI({ temperature: 0.6 }),
  prompt: upgradePrompt,
  outputKey: "upgraded_ideas"
});

// === Agent 3: Scorer ===
const scorerPrompt = new PromptTemplate({
  template: `Score these upgraded ideas objectively:
  Ideas: {upgraded_ideas}
  User Profile: {user_profile}
  Score each on: Originality, Feasibility, Market Potential, Complexity Match. Provide 1-sentence justification for each.`,
  inputVariables: ["upgraded_ideas", "user_profile"]
});
const scorerAgent = new LLMChain({
  llm: new OpenAI({ temperature: 0.2 }),
  prompt: scorerPrompt,
  outputKey: "scored_ideas"
});

// === Sequential Pipeline ===
const ideaGeneratorPipeline = new SequentialChain({
  chains: [inputParser, upgradeAgent, scorerAgent],
  inputVariables: ["raw_input", "user_profile", "original_ideas"],
  outputVariables: ["parsed_input", "upgraded_ideas", "scored_ideas"]
});

// Execute
const result = await ideaGeneratorPipeline.call({
  raw_input: "Students struggle to find internships",
  user_profile: JSON.stringify(userProfile),
  original_ideas: JSON.stringify(retrievedIdeas)
});
```

7. Database Queries & Operations

Query 1: Semantic Search with Platform Filter

```
SELECT id, title, description, innovationScore, marketPotential, complexity, 1 -  
(embedding <=> $1::vector) as similarity FROM "CuratedIdea" WHERE platform = $2 AND  
complexity <= $3 -- User skill based AND  
(suggestedStack->>'requiresPaidAPI')::boolean = false OR $4 > 0 -- Budget check  
ORDER BY (similarity * 0.4 + innovationScore/100 * 0.3 + marketPotential/100 * 0.3)  
DESC LIMIT 10;
```

Query 2: Trending Ideas by Platform (Weekly Update)

```
UPDATE "CuratedIdea" SET trendingScore = (innovationScore * 0.3) + (marketPotential  
* 0.3) + (EXTRACT(DAY FROM NOW() - createdAt) * -0.1) + -- Recency penalty (random()  
* 10) -- Small randomization for discovery WHERE updatedAt < NOW() - INTERVAL '7  
days';
```

Query 3: Get Ideas for Specific User Constraints

```
SELECT * FROM "CuratedIdea" WHERE platform = 'WEB' AND complexity = 'EASY' AND NOT  
(suggestedStack ? 'stripe') -- No paid APIs AND tags && ARRAY['education',  
'students'] ORDER BY innovationScore DESC LIMIT 8;
```

8. API Endpoints

Method	Endpoint	Description	Request Body
POST	/api/generator/parse	Parse user input	{input: string}
POST	/api/generator/retrieve	Search curated DB	{parsedInput, platform, userProfile}
POST	/api/generator/upgrade	Upgrade ideas	{originalIdeas, userProfile, userNeed}
POST	/api/generator/score	Score upgraded ideas	{upgradedIdeas, userProfile}
POST	/api/generator/full	Complete pipeline	{input, platform, userProfile}
GET	/api/curated/platforms	Get all platforms	—
GET	/api/curated/trending	Get trending ideas	{platform?, limit?}
POST	/api/curated/submit	Submit new idea	{idea, source}

9. Interview Talking Points

Architecture Decision: Why Curated Database + AI Upgrade?

Sirf AI se generate karne se generic ideas aate hain. Curated database se quality guaranteed hai — ye hackathon winners, YC startups, aur trending products ke real ideas hain. AI ka kaam hai un ideas ko user ki specific need pe personalize karna, naye ideas banana nahi. Isse quality + personalization dono milta hai.

Why Semantic Search Instead of Keyword Search?

Keyword search mein 'internship' search karne pe sirf 'internship' wale ideas aayenge. Semantic search mein 'students struggle to find internships' se 'job matching', 'career platform', 'skill gap analyzer' bhi match honge — kyunki meaning similar hai. OpenAI embeddings use karte hain for this.

How Does Upgrade Agent Work?

Upgrade agent curated idea ko user profile ke hisaab se modify karta hai. Example: Same 'AI Resume Optimizer' idea, beginner ke liye 'Simple Resume Checker' ban jata hai (no framework, client-side only), advanced ke liye 'AI Career Coach' ban jata hai (GPT-4, vector DB, personalized learning path). Core concept same, implementation different.

Lazy Loading Benefit?

5-8 ideas generate karne mein agar har ek ka blueprint bhi generate karein, toh \$5-10 API cost per user. Lazy loading se sirf selected idea ka blueprint generate hota hai — cost 80% kam. Plus response time fast, kyunki pehle sirf text cards show hote hain.

How Do You Keep Database Fresh?

Weekly cron job se trendingScore update hota hai — Google Trends, GitHub stars, Product Hunt votes se. Community bhi submit kar sakti hai. Plus vector embeddings re-calculate hote hain jab naye ideas add hote hain.

10. Feature Summary

Component	Technology	Purpose
Curated Database	PostgreSQL + pgvector	500+ pre-loaded quality ideas per platform
Semantic Search	OpenAI Embeddings + pgvector	Meaning-based idea retrieval
Input Parser	LangChain LLM (temp: 0.3)	Structured problem extraction
Retriever	Prisma + Raw SQL	Platform-filtered similarity search
Upgrade Agent	LangChain LLM (temp: 0.6)	Personalize ideas for user needs
Scorer Agent	LangChain LLM (temp: 0.2)	Objective 4-dimension scoring
Output	Structured JSON	UI-ready ranked cards
Lazy Loading	On-demand generation	80% cost reduction

— End of Guide 1: Idea Generator PRD —